2024 5th International Conference on Advancements in Computational Sciences (ICACS) | 979-8-3503-9478-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICACS60934.2024.10473272

Enhancing Automated COVID Diagnosis from Chest X-rays using Convolutional Neural Networks and Transfer Learning

Muhammad Abdullah Shah Bukhari
Department of Computer Science
University of the Punjab
Lahore, Pakistan
bcsf19m553@pucit.edu.pk

Muhammad Ahmad Siddiqui
Department of Computer Science
University of the Punjab
Lahore, Pakistan
bcsf19m508@pucit.edu.pk

Sarmad Khalique
Department of Computer Science
University of the Punjab
Lahore, Pakistan
bcsf19m526@pucit.edu.pk

Faisal Bukhari
Department of Data Science
University of the Punjab
Lahore, Pakistan
faisal.bukhari@pucit.edu.pk

Waheed Iqbal
Department of Data Science
University of the Punjab
Lahore, Pakistan
waheed.iqbal@pucit.edu.pk

***Abstract*—We propose an approach for the early detection of COVID-19 and other related lung diseases using artificial intelligence (AI) and deep learning-based methods. The proposed approach involves utilizing transfer learning over convolutional neural networks (CNNs) for the classification of chest X-ray images as normal or COVID-19 positive. To address the limited availability of X-ray images, we employed data augmentation techniques to expand the dataset size and enhance our model's performance on unseen data. We used variations of four pre-trained models: DenseNet, Inception, ResNet, and VGG, and found that DenseNet169 demonstrated the best performance with a validation accuracy of 100%.**

Automated diagnosis, Chest X-rays, CNN, COVID-19, Deep learning, Disease detection, Image processing, Medical image analysis, Transfer learning

I. Introduction

In 2019-2020, a virus called SARS-CoV-2 caused a widespread outbreak known as COVID-19, impacting numerous individuals worldwide. The primary organ affected by this virus was the lungs, leading to difficulties in breathing for those infected. As of August 30, 2023, the reported number of confirmed COVID-19 cases worldwide was approximately 770.085 million [17].

COVID-19 is a virus that emerged as an epidemic in China in December 2019 and has since become a pandemic. Its common symptoms include a dry cough, headache, fever, loss of taste or smell, and shortness of breath. The coronavirus obtained its name due to its characteristic appearance, resembling a halo, when observed under an electron microscope. This virus is primarily transmitted through respiratory droplets and can lead to severe illnesses, including Pneumonia and fatality. The viral genome enters host cells and has a higher incidence in males than females and adults than children. Mortality rates are estimated at 1 to 5% for children. The most common diagnostic method is reverse transcriptase polymerase chain reaction (RT-PCR) [8]. Some of the challenges in dealing with COVID-19 include a long wait period for the test results and a limited number of testing kits. In addition, test sensitivity is a concern due to potential laboratory and sample errors. Other methods, such as computed tomography (CT) and chest x-ray (X-ray) imaging, are also used but require specialist analysis of the images [8] [1].

One of the primary methods of diagnosing COVID-19 is using chest X-ray images. Still, the process of reading and interpreting these images can be time-consuming and less accurate without the knowledge and experience of specialists. The challenge is that other medical conditions, such as lung cancer or excess fluid, can also show similar anomalies on X-ray images. Therefore, an accurate and efficient method for the early identification of chest-related diseases is urgently needed [6].

In recent years, the use of technology has drastically increased in many fields. From farming to launching

979-8-3503-9478-8/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

space crafts, in almost every area, AI is widely used. The use of AI has also affected the healthcare sector with several advancements. It is being commonly used in the field of medicine [5]. With the help of AI and the use of applications like Image Processing, we can detect different abnormalities in X-rays and can reduce the effort and time being put in by a specialist to diagnose a disease.

Researchers have proposed various computational algorithms and supporting diagnostic tools for analyzing X-ray images. However, these tools may not provide enough information to help doctors make decisions. Machine learning is a promising approach in this area, but traditional methods such as vector quantization and recurrent neural networks have limitations in terms of accuracy and computation time [8] [1] [6].

Given these limitations, researchers are exploring using artificial intelligence (AI) and deep learning (DL)-based methods for the early detection of COVID-19 and other related lung diseases. These methods can help specialists make quick and accurate diagnoses, speeding up the treatment process. Deep learning methods, in particular, have gained widespread popularity in this field because they eliminate the need for feature extraction during the preprocessing stage. Instead, they can learn the most relevant features directly from the used dataset.

In the case of chest X-ray images, deep learning models process the data through multiple layers, each containing numerous nodes. These nodes perform feature extraction tasks, and the activation function determines whether to activate the next layer based on the results. The flexibility of deep learning allows for different numbers of nodes in each layer, depending on the specific task at hand.

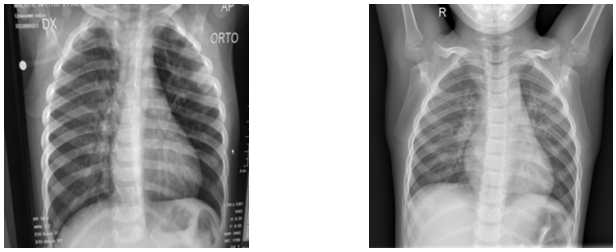
II. Materials, Methodology, and Preprocessing

Through the course of this segment, we will explain the algorithm we used for COVID-19 detection and the dataset on which we trained our model.

A. Dataset

The dataset we gathered was from two unique sources. JP Cohen's repository was used to collect 142 COVID-19 positive images [2] while 142 normal images were collected from Kaggle [7]. An equal number of COVID-19 and Normal chest X-ray images were collected to address the class imbalance problem. We used 80% of our data (or 224 images) for training, while 20% of our data (or 60 images) were used for validation.

While the dataset size is relatively modest, it was sourced from JP Cohen's renowned repository, encompassing both COVID-19-positive and normal images. Figure 1 shows both a normal and Covid-19 positive lung side by side. A majority of the studies against which we benchmarked our results utilized a comparable dataset . Examining these benchmark studies reveals that most of them incorporated datasets with fewer than 200 images, encompassing both normal and COVID-19-positive instances. Even in cases where the dataset size exceeded 500 images, such as the study by Ozturk et al., which included 625 images, the distribution remained imbalanced with 125 COVID-positive and 500 normal images. The inherent imbalance in dataset distribution gives rise to a class imbalance issue, wherein the model exhibits bias, demonstrating superior recognition performance for the class with a larger dataset. Consequently, in this context, only the 125 normal images are effectively utilized, rendering the remaining 375 normal images redundant. To rectify this class imbalance, achieving parity between the number of COVID and normal images at 125 instances each becomes imperative.



Covid Normal

Fig. 1: Posterior Anterior view of a Covid-19 positive (left) and normal lung (right)

B. Suggested method

The suggested approach for detecting COVID-19 is explained in Figure 2 . The model aims to output the probability of COVID-19. This encompasses two crucial phases: preprocessing and classification using Transfer Learning over CNNs. A comprehensive account of each stage is provided in the following segments.

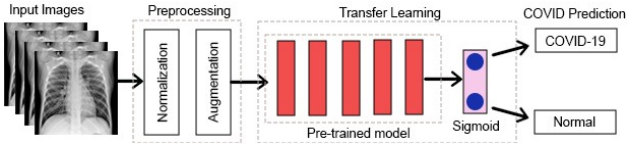


Fig. 2: Proposed Architecture

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

C. Preprocessing

In this segment, we will go through the two techniques we used for preprocessing.

1) Preprocessing Stage 1: Normalization : Data normalization is commonly employed in CNN-based architectures to ensure numerical stability. By normalizing the data, the CNN model is expected to exhibit faster learning and more stable gradient descent [13]. In this study, the pixel values of the input images were normalized to a range of 0-1. The dataset used consisted of grayscale images. The normalization was done by multiplying the pixel values by 1/255.

2) Preprocessing Stage 2: Data Augmentation : CNNs require large amounts of training data to learn and generalize well on new and unseen data, something which was not possible with our available dataset of 284 images. To address this issue, we employed data augmentation techniques that not only expanded the dataset size but also enhanced our model's performance on unseen data.

The following tools were used for performing augmentation:

1. The images were subjected to a shear range of up to 0.2.
2. The images were subjected to a zoom range of up to 0.15.
3. The images were subjected to a horizontal flip.
4. The images were subjected to a width shift range of up to 0.2.
5. The images were subjected to a height shift range of up to 0.2.
6. The images were subjected to a rotation of up to 20 degrees.
7. The images were subjected to a channel shift range of up to 20.
8. The images were subjected to a brightness adjustment factor between 0.5 and 1.5.

Ultimately, a significantly bigger training set comprising 1808 images was acquired, constituting an eight-fold increase compared to the original training images.

Figure 3 shows each of the 8 data augmentation techniques applied.

Figure 4 shows the effect of data augmentation on our original image.

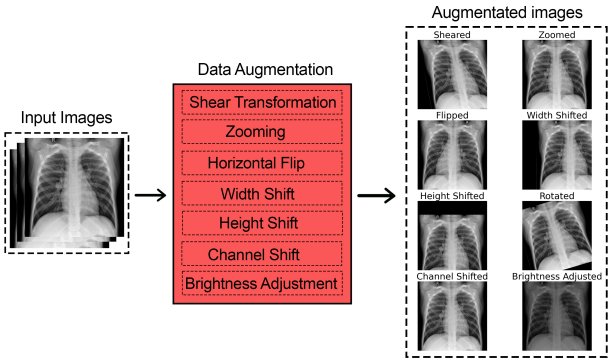


Fig. 3: All augmentation techniques

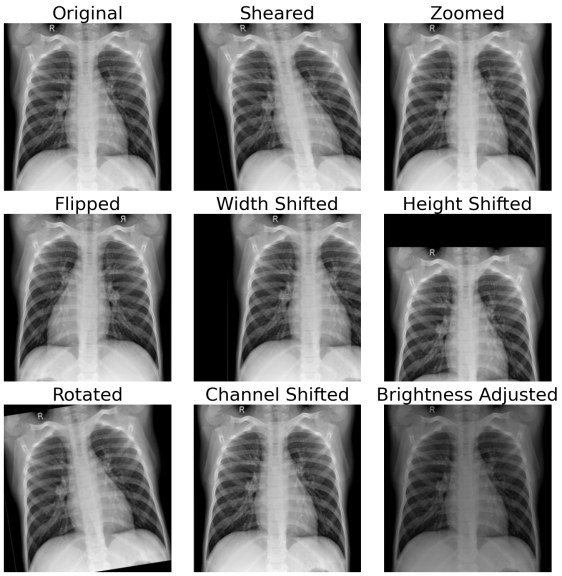


Fig. 4: Demonstration of Data Augmentation results

D. Approach: Utilizing Pre-trained CNN models for COVID-19 prediction

CNNs have generally been shown to perform exceptionally well on image classification tasks. However, they require a large training dataset to generalize well on unseen data, which is not possible due to the limited number of X-ray images. To deal with this, we used variations of 4 pre-trained models: (1) DenseNet, (2) Inception, (3) ResNet, and (4) VGG. These models were chosen as they have shown excellent results, particularly in the domain of medical images and X-ray image classification. We altered these pre-trained models to fit our use case of detecting COVID-19. We removed the last layer of the pre-trained models and added four extra

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

fully connected layers. These layers had 512, 256, 128, and 1 neuron respectively. The final FC layer consisted of 1 neuron as we were doing binary classification. To take advantage of the pre-trained weights, only the last four layers were set as trainable. The other layers were instantiated with pre-trained weights from Imagenet. The same hyperparameters were used across all the pre-trained models for a balanced comparison. Figure 5 outlines the architecture of our model.

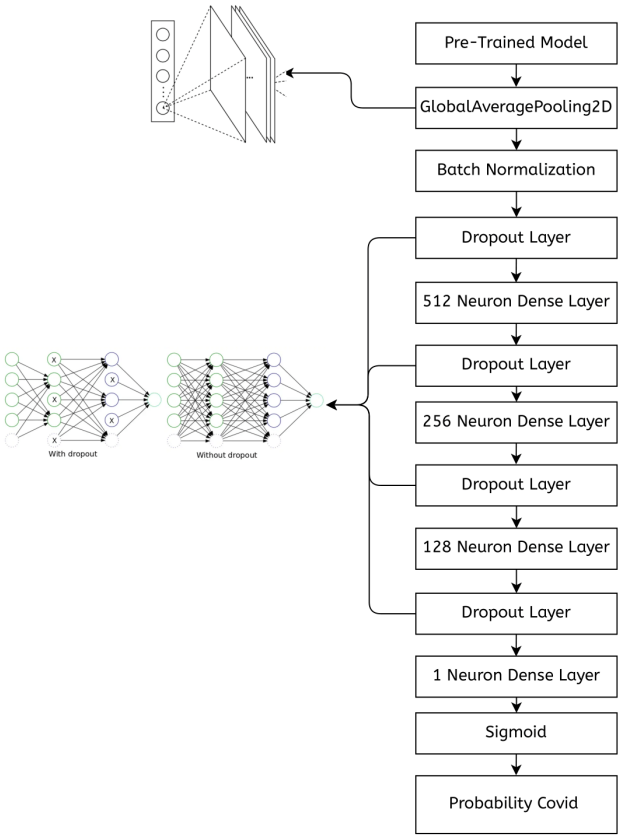


Fig. 5: Architectural Comparison

The architectural overview of DenseNet-121, DenseNet-169 and DenseNet-201 is as follows:

DenseNet-121 is a convolutional neural network (CNN) that has undergone training on the ImageNet dataset. Its architectural design encompasses multiple dense blocks, transition layers, and fully connected layers.

DenseNet-121 initiates with a 7×7 convolutional layer, followed by batch normalization and ReLU activation. Comprising four dense blocks, each block contains densely connected layers. The output from every layer within a block is concatenated with the outputs from all preceding layers, enabling the network to harness features from all layers for potential performance enhancement.

Between the dense blocks, transition layers are introduced to reduce spatial dimensions and the number of channels, preventing the network from becoming excessively large and computationally demanding. Following the final dense block, a global average pooling layer aggregates spatial information across the feature map and condenses the dimensions to a fixed size.

Subsequently, a fully connected layer with 1,000 neurons, aligning with ImageNet categories, is employed. The softmax activation function is then applied to obtain the ultimate class probabilities. DenseNet-169 and DenseNet-201 represent deeper variations of DenseNet-121, featuring six dense blocks instead of four, and thus enabling them to capture more intricate patterns. However, this also results in an increased number of parameters (14.3 million and 20 million, respectively) and greater computational resource requirements for training and inference.

Moving to InceptionV3, its architectural overview entails a series of convolutional layers that initially reduce the input image's dimensionality. These layers utilize a combination of 3×3 and 7×7 filters with stride 2, as well as max pooling and batch normalization layers.

InceptionV3 employs multiple stacked Inception modules, with each module comprising parallel branches utilizing different convolution kernel sizes, including 1×1, 3×3, and 5×5 convolutions, alongside max pooling operations. This enables the network to learn features across various spatial scales. Reduction modules are introduced periodically to downsample feature maps using a combination of 1×1 convolutions, 3×3 convolutions, and max pooling operations.

InceptionV3 also integrates two auxiliary classifiers, additional branches designed to offer intermediate supervision during training. These classifiers encourage the network to acquire more discriminative features and mitigate the vanishing gradient problem. Following the last inception module, a global average pooling layer aggregates spatial information across the feature map and reduces the dimensions to a fixed size. A fully connected layer with 1,000 neurons, matching ImageNet categories, applies the softmax activation function to determine final class probabilities.

Transitioning to VGG16 and VGG19, VGG16 consists of 13 convolutional layers followed by three fully

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

connected layers. These convolutional layers primarily use 3×3 filters, with a few 1×1 filters, followed by ReLU activation functions and 2×2 max pooling operations for spatial dimension reduction. The fully connected layers contain 4,096 neurons each and are also followed by ReLU activation functions. The final fully connected layer consists of 1,000 neurons, matching ImageNet categories, and utilizes a softmax activation function to obtain class probabilities.

VGG19 closely resembles VGG16 but incorporates an additional three convolutional layers, totaling 19 layers. These extra layers, introduced in the middle of the network, allow for deeper feature extraction. The additional layers include two 3×3 convolutional layers and one 1×1 convolutional layer, enhancing the model's representational power by capturing more intricate patterns.

Finally, examining ResNet50, ResNet101, and ResNet152, they are part of the ResNet (Residual Network) family, known for their skip connections and residual blocks, addressing deep neural network degradation. ResNet50, trained on ImageNet, starts with a 7×7 convolutional layer and includes multiple residual blocks, each with a sequence of convolutional layers and skip connections. After the final residual block, a global average pooling layer reduces dimensions, followed by a fully connected layer with 1,000 neurons and softmax activation for class probabilities.

ResNet101 and ResNet152 are deeper counterparts, with 101 and 152 layers, offering potential advantages in capturing complex features but requiring more parameters and computational resources. ResNet101 comprises 44.6 million parameters, while ResNet152 includes 60.3 million, compared to ResNet50's 25.6 million parameters.

III. Experiments and Their Findings

In this segment, we will showcase different TL models we used and their performances. Finally, we will compare the results of our best-performing TL model with those of existing top-notch approaches. The TL models we used for the experimentation are as follows:

1. DenseNet121
2. DenseNet169
3. DenseNet201
4. VGG16
5. VGG19
6. InceptionV3
7. Resnet50
8. Resnet101
9. Resnet152

A. Experimental configuration and assessment metrics used

We assessed our approach using the dataset collected from JP Cohen's repository [2] and the Kaggle dataset [7]. As mentioned earlier, we used eight augmentation techniques which gave us a combined dataset of 1852 images. Of these 1852 images 60 were used for validation while 1792 were used for training.

The input images were resized to a dimension of 224×224 while using all of the eight different TL models. The algorithm we used was built using the Functional API of TensorFlow. Furthermore, we set the batch size to 8 and the number of epochs to 15 across all TL models. We used Adams as our optimizer with a learning rate of 0.01. We used binary cross entropy as our loss function, whose formula is given below.

$$\text{Binary Cross-Entropy} = -[y \log(p) + (1-y) \log(1-p)] \tag{1}$$

Here y represents the true binary label (0 or 1). p represents the predicted probability of the positive class. In the formula, the first term - y * log(p) calculates the loss when the true label is 1, and the second term - (1 - y) * log(1 - p) calculates the loss when the true label is 0. By summing these two terms and taking the negative, you get the overall binary cross-entropy loss.

We chose these particular values of the hyperparameters as they were giving us the best results (results shown later on). The metrics used to gauge the performance of the TL models are listed as follows:

1. Recall
2. Precision
3. F1-Score
4. Accuracy
5. Area Under the Curve (AUC)

The **Recall** measures the ability of a model to correctly identify the positive cases out of all the actual positive cases. Here TP denotes true positive, and FN denotes False Negative.

The **Precision** measures the percentage of correctly predicted positive cases out of all cases predicted as positive. Here TP denotes True Positive, and FP denotes False Positive.

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

The **Specificity** measures the model's ability to identify negative cases out of all actual negative cases correctly. Here TN denotes True Negative, and FP denotes False Positive.

The **F1-score** combines Precision and recall into a single value to give the best balance between them both. A higher F1 score denotes better performance.

Accuracy measures the overall correctness of a model's predictions.

The **AUC** (Area Under the ROC Curve) measures the model's ability to discriminate between positive and negative cases across various classification thresholds. Here the TPR(T) represents the True Positive Rate at a given classification threshold T. It measures the proportion of correctly identified positive cases (Recall). FPR'(T) represents the Negative Positive Rate at a given classification threshold T. It measures the proportion of incorrectly identified negative instances (1 - Specificity). I(T' ¿ T) is an indicator function that checks if T' (a different classification threshold) is greater than T.

f1(T') and f0(T) are probability density functions representing the distributions of positive and negative cases, respectively. A higher AUC denotes better performance.

The formulas for the different metrics are listed below:

$$Recall = \frac{TP}{TP + FN} \tag{2}$$

$$Precision = \frac{TP}{TP + FP} \tag{3}$$

$$Specificity = \frac{TN}{TN + FP} \tag{4}$$

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{5}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{6}$$

$$\text{AUC} = \frac{\int_{-\infty}^{\infty} \text{TPR}(T) \cdot (-\text{FPR}'(T))\, dT}{\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(T' > T) \cdot f_1(T') \cdot f_0(T)\, dT'\, dT} \tag{7}$$

Class	Original Dataset		Augmented Dataset	
	Train	Val	Train	Val
COVID-19	112	30	896	30
Normal	112	30	896	30
Total	224	60	1792	60

TABLE I: Distribution of Dataset before and after Augmentation

B. Results

Table I shows the size of the dataset before and after data augmentation was applied.

Table II shows the performance of the nine different TL models used by comparing validation loss and validation accuracy.

It can be shown that DenseNet169 demonstrated the best performance with a validation accuracy of 100% and a validation loss of 0.0492. The unexpectedly high validation accuracy can be attributed to the fact that we performed augmentation only on the training data and not the validation data. This would have made the validation data easier to predict than the training data.

The detailed comparison of the models is shown in Table III in terms of the six metrics we discussed earlier. It can be seen that DenseNet169 performed the best with a precision of 1, an F1-Score of 1, an AUC of 1, a Recall of 1, and a validation accuracy of 100. It is shown that the Recall of all models apart from the Resnet152 was 100%.

Model	Val_Acc	Val_Loss
InceptionV3	96.67	0.0744
Densenet121	98.33	0.0518
Densenet169	100	0.0492
Densenet201	98.33	0.0959
VGG16	98.33	0.2783
VGG19	85	0.3037
Resnet50	71.67	0.4969
Resnet101	88.33	0.6429
Resnet152	51.67	0.7848

TABLE II: A High-Level Performance Comparison of All Models

Model	Pre	Specificity	F1-Score	AUC	Rec	Val_Acc
InceptionV3	0.9091	1	0.9474	0.9933	1	96.67
Densenet121	0.9375	1	0.9677	1	1	98.33
Densenet169	1	1	1	1	1	100
Densenet201	0.9667	1	0.9818	0.9989	1	98.33
VGG16	0.8571	1	0.9831	0.9956	1	98.33
VGG19	0.7692	1	0.8684	0.9956	1	85
Resnet50	0.6383	1	0.7788	0.9956	1	71.67
Resnet101	0.9259	0.9667	0.875	0.9639	0.8333	88.33
Resnet152	0.5	0.9667	0.6667	0.9222	1	51.67

TABLE III: Classification results in a comparison of all nine CNN models.

1) Analyzing Results from Different Optimizers (For Covid): This sub-segment analyzes the performance of our top 2 models with four different optimizers (1) Adadelta, (2) SGD, (3) RMSProp, and (4) Adam. Three unique learning rates (0.1, 0.01, 0.001) were used to tune the performance of the optimizers.

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

Optimizer	Precision	Sensitivity / recall	Specificity	Val_Accuracy	AUC	F1-Score
Adadelta						
0.1	0.9375	1	1	0.9667	0.9978	0.9677
0.01	0.7931	0.7667	0.9000	0.7833	0.8850	0.7794
0.001	0.5357	1	0.7333	0.5667	0.6861	0.6967
SGD						
0.1	1	1	1	1	1	1
0.01	1	0.9667	1	0.9833	1	0.9831
0.001	0.7143	1	1	0.8000	0.9922	0.8333
RMSProp						
0.1	1	1	1	1	1	1
0.01	1	1	1	1	1	1
0.001	1	0.9667	1	0.9833	0.9978	0.9828
Adam						
0.1	1	0.9667	1	0.9833	1	0.9831
0.01	1	0.9667	1	0.9833	0.9978	0.9831
0.001	1	0.9667	1	0.9833	1	0.9831

TABLE IV: Performance of DenseNet121

Optimizer	Precision	Sensitivity / recall	Specificity	Accuracy	AUC	F1-Score
Adadelta						
0.1	1	0.9667	1	0.9833	1	0.9828
0.01	0.5000	0.0333	0.4333	0.5000	0.4544	0.0638
0.001	0.6333	0.6333	0.8000	0.6333	0.7256	0.6333
SGD						
0.1	1	0.9667	1	0.9833	1	0.9828
0.01	1	1	1	1	1	1
0.001	1	0.9667	1	0.9833	0.9989	0.9828
RMSProp						
0.1	1	0.9667	1	0.9833	0.9833	0.9828
0.01	1	1	1	1	1	1
0.001	1	0.9667	1	0.9833	1	0.9828
Adam						
0.1	1	1	1	1	1	1
0.01	1	0.9667	1	0.9833	1	0.9828
0.001	1	0.9667	1	0.9833	0.9994	0.9828

TABLE V: Performance of DenseNet169

Optimizer	Precision	Sensitivity / recall	Specificity	Accuracy	AUC	F1-Score
Adadelta						
0.1	0.9667	0.9667	1	0.9667	0.9978	0.9667
0.01	0.8438	0.9000	0.9667	0.8667	0.9256	0.8713
0.001	0.6667	0.1333	0.7333	0.5333	0.7206	0.2222
SGD						
0.1	1	1	1	1	1	1
0.01	0.9667	0.9667	1	0.9667	0.9983	0.9667
0.001	0.8333	1.	1	0.9000	0.9933	0.9091
RMSProp						
0.1	1	0.9667	1	0.9833	0.9833	0.9828
0.01	1	0.9667	1	0.9833	0.9833	0.9828
0.001	1	1	1	1	1	1
Adam						
0.1	1	1	1	1	1	1
0.01	1	1	1	1	1	1
0.001	1	0.9667	1	0.9833	1	0.9828

TABLE VI: Performance of DenseNet201

Optimizer	Precision	Sensitivity / recall	Specificity	Accuracy	AUC	F1-Score
Adadelta						
0.1	0.9655	0.9333	1	0.9500	0.9889	0.9487
0.01	1	0.2333	0.9333	0.6167	0.9006	0.3750
0.001	0.5000	1	0.3667	0.5000	0.3700	0.6667
SGD						
0.1	0.9667	0.9667	1	0.9667	0.9967	0.9667
0.01	0.9333	0.9333	1	0.9333	0.9872	0.9333
0.001	0.9615	0.8333	1	0.9000	0.9733	0.8923
RMSProp						
0.1	1	0.9667	1	0.9833	0.9811	0.9831
0.01	1	1	1	1	1	1
0.001	0.8571	1	1	0.9167	0.9978	0.9231
Adam						
0.1	0.8788	0.9667	1	0.9167	0.9911	0.9206
0.01	0.9375	1	1	0.9667	0.9989	0.9677
0.001	0.9667	0.9667	1	0.9667	0.9872	0.9667

TABLE VII: Performance of VGG16

Table IV shows the performance of DenseNet121, Table V shows the performance of DenseNet169, Table VI shows the performance of DenseNet201 and Table VII shows the performance of VGG16

2) Evaluating Performance With Varying Batch Sizes: Selecting an appropriate batch size is highly important when tuning CNN models. Table VIII shows the validation accuracy when using batch sizes 8, 16, and 32. It can be seen that a batch size of 8 performed better for most TL models apart from InceptionV3, Resent50, and DenseNet201.

Model	Batch = 8	Batch = 16	Batch = 32
DenseNet169	100	98.33	98.33
DenseNet121	100	98.33	93.33
DenseNet201	98.33	100	98.33
ResNet101	96.67	90	53.33
ResNet50	66.67	50	76.67
Resnet152	65	50	50
VGG16	98.33	96.67	88.33
VGG19	90	50	50
InceptionV3	95	96.67	96.67

TABLE VIII: Results with different batch sizes

3) Comparison with state-of-the-art deep learning COVID-19 detection approaches using chest X-ray images: In this subsection, we compare the outcomes obtained by the top-performing CNN model with our recently proposed approach for COVID-19 detection. Table IX shows our approach beating all the other state-of-the-art approaches by having a validation accuracy of 100%.

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

TABLE IX: Comparison with State of the Art Methods

Reference	Methodology	Classes	Samples Count	Accuracy (%)
Hemdan et al. [4]	COVIDX-Net	COVID-19	50	90.00
–	–	Normal	C: 25, N: 25	–
Narin et al. [9]	ResNet-50	COVID-19	100	98.00
–	–	Normal	C: 50, N: 50	–
Sethy and Behera [12]	ResNet-50 and SVM	COVID-19	50	95.38
–	–	Normal	C: 25, N: 25	–
Toğaçar et al. [14]	SqueezeNet and MobileNetV2	COVID-19	458	98.25
–	SMO and SVM	Normal	C: 295, N: 65, P: 98	–
–	–	Pneumonia	–	–
Wang and Wong [16]	COVID-Net	COVID-19	13800	92.60
–	–	Normal	C: 183, N: –, P: –	–
–	–	Pneumonia	–	–
Ucar and Korkmaz [15]	Bayes-SqueezeNet	COVID-19	5949	98.30
–	–	Normal	C: 76, N: 1583, P: 4290	–
–	–	Pneumonia	–	–
Farooq and Hafeez [3]	COVID-ResNet	COVID-19	5941	96.23
–	–	Normal	C: 68, N: –, BP: –, VP: –	–
–	–	Bacterial pneumonia	–	–
–	–	Viral pneumonia	–	–
Ozturk et al. [11]	DarkCovidNet	COVID-19	625	98.08
–	–	Normal	C: 125, N: 500	–
–	–	COVID-19	1125	87.02
–	–	Normal	C: 125, N: 500, P: 500	–
–	–	Pneumonia	–	–
Soumya et al. [10]	ResNet-34	COVID-19	406	98.33
Proposed Method	DenseNet169	Normal COVID-19	C: 142, N: 142	100

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.

IV. Conclusion

In this paper, we attempted to propose an effective Covid-19 detector. Our results showed that the DenseNet-169 was the best-performing model with a validation accuracy of 100% on the COVID-19 dataset. Our results were shown to be better than the state-of-the-art approaches which achieved an accuracy of 98.33% at max. Our results do not guarantee a 100% accuracy but they do show that our approach performs better on a common dataset utilized by different state-of-the-art approaches.

References

[1] Gaffari Celik. Detection of covid-19 and other pneumonia cases from ct and x-ray chest images using deep learning based on feature reuse residual block and depthwise dilated convolutions neural network. *Applied Soft Computing*, 133:109906, 2023.

[2] Joseph Paul Cohen, Paul Morrison, and Lan Dao. Covid-19 chest x-ray dataset. https://github.com/ieee8023/covid-chestxray-dataset, 2021.

[3] Muhammad Farooq and Abdul Hafeez. Covid-resnet: A deep learning framework for screening of covid19 from radiographs, 2020.

[4] Ezz El-Din Hemdan, Marwa A. Shouman, and Mohamed Esmail Karar. Covidx-net: A framework of deep learning classifiers to diagnose covid-19 in x-ray images, 2020.

[5] Steven A Hicks, Inga Strümke, Vajira Thambawita, Malek Hammou, Michael A Riegler, Pål Halvorsen, and Sravanthi Parasa. On evaluation metrics for medical applications of artificial intelligence. *Scientific Reports*, 12(1):1–9, 2022.

[6] Amit Kumar Jaiswal, Prayag Tiwari, Sachin Kumar, Deepak Gupta, Ashish Khanna, and Joel J.P.C. Rodrigues. Identifying pneumonia in chest x-rays: A deep learning approach. *Measurement*, 145:511–518, 2019.

[7] Paul Mooney. Chest x-ray images (pneumonia). https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia, 2022.

[8] Zohreh Mousavi, Nahal Shahini, Sobhan Sheykhivand, Sina Mojtahedi, and Afrooz Arshadi. Covid-19 detection using chest x-ray images based on a developed deep neural network. *SLAS Technology*, 27(1):63–75, 2022.

[9] Ali Narin, Ceren Kaya, and Ziynet Pamuk. Automatic detection of coronavirus disease (covid-19) using x-ray images and deep convolutional neural networks. *Pattern Analysis and Applications*, 24(3):1207–1220, May 2021.

[10] Soumya Ranjan Nayak, Deepak Ranjan Nayak, Utkarsh Sinha, Vaibhav Arora, and Ram Bilas Pachori. Application of deep learning techniques for detection of covid-19 cases using chest x-ray images: A comprehensive study. *Biomedical Signal Processing and Control*, 64:102365, 2021.

[11] Tulin Ozturk, Muhammed Talo, Eylul Azra Yildirim, Ulas Baran Baloglu, Ozal Yildirim, and U. Rajendra Acharya. Automated detection of covid-19 cases using deep neural networks with x-ray images. *Computers in Biology and Medicine*, 121:103792, 2020.

[12] Pradeep Kumar Sethy and Somanath K. Behera. Detection of coronavirus disease (covid-19) based on deep features. *Preprints.org*, 2020(3):300, 2020.

[13] Zar Nawab Khan Swati, Qinghua Zhao, Muhammad Kabir, Farman Ali, Zakir Ali, Saeed Ahmed, and Jianfeng Lu. Brain tumor classification for mr images using transfer learning and fine-tuning. *Computerized Medical Imaging and Graphics*, 75:34–46, 2019.

[14] Mesut Toğaçar, Burhan Ergen, and Zafer Cömert. Covid-19 detection using deep learning models to exploit social mimic optimization and structured chest x-ray images using fuzzy color and stacking approaches. *Computers in Biology and Medicine*, 121:103805, 2020.

[15] Ferhat Ucar and Deniz Korkmaz. Covidiagnosis-net: Deep bayes-squeezenet based diagnosis of the coronavirus disease 2019 (covid-19) from x-ray images. *Medical Hypotheses*, 140:109761, 2020.

[16] Linda Wang and Alexander Wong. Covid-net: A tailored deep convolutional neural network design for detection of covid-19 cases from chest x-ray images, 2020.

[17] World Health Organization. Covid-19 dashboard, Year. Accessed on Month Day, Year.

Authorized licensed use limited to: University of Punjab. Downloaded on August 25,2024 at 10:59:43 UTC from IEEE Xplore. Restrictions apply.